WEEK 9 SESSION 2

From Exercises to a Solo Project

Topic selection, requirements analysis and personal project planning
120 minutes including a break / Instructor: MinGi KYUNG

01. The Solo Term Project

Weeks 9–16: one student, one independently developed program
What is graded and what “done” means

The toolkit is complete

Weeks	You learned	You built
1–2	Variables, types, I/O, operators, strings, if	BMI calculator
3–4	Loops, functions, lists, dicts, sets	Grade manager, word counts
5–6	Modules, exceptions, files, classes	Lotto, journal, charts
7	OOP four principles, pandas, matplotlib	Bank accounts, CSV reports
8	Review, Git basics: add, commit, push	Midterm practical

Nothing new is required to start the project.

The project is where these pieces solve one real problem together.

What changes today

Coursework so far	Solo term project
Given TODOs	You choose the problem and scope
One exercise file	Several files forming one program
A known answer	You write acceptance criteria
Short individual exercises	An individual project maintained over several weeks
Automatic checks	Evidence from planning, design, code, documentation and demo

Project rule: individual projects only. Each student owns and explains the work.

The eight-week roadmap

Wk	Focus	Deliverable
9	Planning	Personal profile, topic worksheet, proposal v0
10	System design and Git workflow	Design document, own repository and feature branch
11	Prototyping and MVP	Core scenario runs end to end
12	Testing and debugging	Acceptance tests and bug fixes
13	Data or UI extension	Selected extra features and integrated data
14	Documentation and freeze	README, final cleanup, no new features
15	Rehearsal	Slides and timed demo
16	Final presentation	Individual demo, code and explanations

Show concrete progress every week.

How the solo project is graded

Component	%	Evidence
Planning and requirements	15%	Personal proposal and design document
Design and Git workflow	20%	Structure, meaningful commits and branch history
Implementation	30%	Running program and code quality
Documentation	15%	README with reproducible setup and usage
Presentation and demo	20%	Individual demonstration and answers

The original project weights stay the same; no teamwork score applies.

Commit history documents your own development process.

Individual ownership

Solo projects only: one student submits one project.

You own planning, implementation, data, tests, documentation and the demo.

Keep your own repository and record your own progress.

Advice and peer feedback can improve decisions; implementation remains yours.

A shared implementation or divided development task is not a solo project.

Explain every submitted function and design decision.

Definition of done

Your individual project is complete when it has:

1. A program that runs in a fresh environment using the repository instructions

2. A README explaining its purpose, setup, usage and limitations

3. A reliable individual live demo in week 16

4. Clear answers explaining your implementation and design choices

Freeze features in week 14 and rehearse in week 15.

02. Choosing a Topic

Three tests, one catalog, five red flags

Three tests for a solo topic

1. Care test: you or someone you know actually needs the program.
2. Python test: its core uses logic and data skills you have learned.
3. MVP deadline test: you can run a minimal version by week 11 on your own.

If one test fails, narrow the scope.

If two tests fail, choose another topic.

Include testing and documentation in your available time.

Small is beautiful: the MVP ladder

MVP = the smallest program that does one useful thing end to end.

Level	Idea
Too big	"A fitness app with login, feed, and recommendations"
Still big	"A workout tracker with charts and goals"
MVP	"Append sets to workout.csv, print a weekly summary chart"
Too small	"A script that prints one number" (no user, no decision)

MVP first, then climb: extras are weeks 12–13 work.

Topic catalog: data projects

Core skills: pandas, matplotlib (weeks 5–7)

Study-time logger – log hours per course, weekly bar chart

Weather comparison – two cities' CSV, min/mean/max lines

Personal finance – expenses.csv, category pie and monthly trend

Campus data – cafeteria menus, library hours, exam-period demand

Data must exist or be collectable by hand within days.

Topic catalog: tools and games

Core skills: functions, classes, files (weeks 3–7)

Vocabulary flashcards – CSV word list, quiz mode, wrong-answer review

Word game clone – Wordle-style, one class per round

Inventory manager – add/remove/search, saved to a file

Password generator + strength checker

Quiz builder – teachers write questions, students play, scores saved

A game needs a clear "one round" you can demo.

Topic catalog: everyday helpers

Core skills: dictionaries, classes and exceptions

Timetable checker: courses, rooms and conflict warnings

Expense splitter: trip costs and each person’s settlement amount

Recipe planner: a weekly menu and CSV shopping list

Personal task planner: deadlines, priorities and an overdue list

Multiple users may use a program developed by one student.

Red flags

Warning	Risk	Response
“An app” without a user or situation	Unclear scope and demo	Name a user and a moment of use
Unavailable or restricted data	MVP may stall	Use public data or collect a small sample
A new framework is essential	Too much unfamiliar work	Start with console, files and charts
“AI will do the difficult parts”	You cannot explain the code	Explain and verify every submitted part
Scope needs several developers	Too large for one student	Cut to one core scenario

Choose a topic you can implement, test and explain independently.

03. Requirements Analysis

From vague wishes to testable statements

A requirement is testable

Not a requirement	Requirement
"The report is fast"	"The summary prints in under 1 second for 10,000 rows"
"Easy to use"	"A new user can log an entry with one command and no manual"
"Handles bad input"	"Entering a negative amount shows a warning and nothing is saved"

Functional requirements: what the program does.

Non-functional requirements: how well it does it (speed, safety, readability).

If you cannot imagine a test for it, it is not a requirement yet.

User stories

Format: As a ___, I want ___, so that ___.

Story	Quality
As a student, I want to log study time by course so I can review my week.	Clear user, action and reason
As a user, I want features so I can do things.	Too vague
I want to use classes.	A technical choice, not a user need

Write 3–5 stories and connect each to numbered features.

From stories to features

Story: As a student, I want to log study time by course so I can review my week.

ID	Feature	Course skills
F1	Enter date, course and positive minutes	input, conditions, exceptions
F2	Save and reload study.csv	files, CSV, type conversion
F3	Show weekly totals by course	dictionaries and loops
F4	Draw a weekly bar chart	matplotlib

Rejecting invalid minutes is part of F1’s acceptance criteria.

Predict: Which features are essential for the first working version?

MoSCoW prioritization

Priority	Meaning	Study logger
Must	Required for the core demo	F1 valid input, F2 persistence, F3 totals
Should	Important but not essential	F4 chart
Could	Add only if time remains	Course colors
Won't	Explicitly outside this project	Login, mobile app, database

Build the MVP from Must features and run it by week 11.

Solo scope: at most 3 Must features, each with a testable result.

Include validation and error handling inside the relevant feature.

The scope border

In v1	Out of v1 (and where it went)
Log, save, weekly totals	Login / Won't
One chart, one CSV	Database / Won't
Console interface	Web page / Won't this term

The Out column is a decision record. Reviewers will ask what you cut and why.

"Won't" is a plan, not a failure.

Data design: four questions

1. What data does the program read and write?
2. Who creates it: the user, you, or an existing public source?
3. What format and types: file names, columns and value rules?
4. How much: start with a small sample you have checked.

```
study.csv: date,course,minutes
2026-10-24,python,90
2026-10-25,stats,45
```

Specify the data format now, even if integration happens later.

Acceptance criteria

Each Must feature gets at least one pass/fail line.

Feature	Acceptance criterion
F1 log entry	Entering 2026-10-24, python, 90 appends one row to study.csv
F1 bad input	Entering -30 minutes prints a warning; file unchanged
F3 totals	For the sample week above, totals print python 90, stats 45

These lines become week-12 test cases, almost word for word.

Requirements checklist

- ☐ 3–5 user stories naming a user, action and reason
- ☐ Every story connected to numbered features
- ☐ All features labeled Must / Should / Could / Won't
- ☐ At most 3 Must features for the solo MVP
- ☐ At least one acceptance criterion per Must feature
- ☐ Invalid-input behavior specified where relevant
- ☐ Named data files with columns and types
- ☐ Written out-of-scope decisions

04. Planning Your Work

Proposal, milestones, work categories and risks

The solo proposal

Use the solo template; target two pages plus detailed appendices.

Part	Content
1. Problem and goal	Who needs what change?
2. User and situation	One user and moment of use
3. Requirements	Stories, features, priorities and acceptance criteria
4. Scope	Included and excluded features
5. Data	Sources, files, columns and types
6. Skills	Course concepts used by each feature
7. Milestones	Personal weekly results and work blocks
8. Risks	Three major risks and responses

Template: labs/session2-solo/proposal-template.md

Personal milestones

Week	Completion evidence
10	Design document and own repository with a feature branch
11	All Must features run end to end in an MVP demo
12	Acceptance tests pass; remaining bugs prioritized
13	Selected extensions integrated without breaking the MVP
14	Features frozen and README complete
15	Timed rehearsal on the demo machine
16	Individual final presentation

A milestone needs evidence you can show.

One developer, several responsibilities

Work category	Your responsibility
Planning	Scope, schedule and progress log
Implementation	Code structure and working features
Data and testing	Sample data, test cases and bug tracking
Documentation and demo	README, slides and rehearsal

These are work categories for one person, not roles to divide among people.

Reserve time for tests and documentation as well as coding.

Solo-project risks and mitigation

Risk	Early signal	Response
Data too large or dirty	Sample will not load	Use a checked 100-row sample
Personal time overload	One task slips repeatedly	Cut scope and keep one next task
Feature creep	New Must features appear late	Freeze Must scope after week 10
Demo failure	Setup works only on your laptop	Rehearse in the demo environment

Record your own three largest risks and a fallback for each.

A personal weekly routine

Reserve a regular 30-minute progress review.

Write: completed / next / blocked.

Choose one concrete next task and schedule its work block.

Keep meaningful commits for code, data and documentation.

Use Markdown, your own Git repository and the LMS submission channel.

Record your weekly review and coding times today.

Today’s individual deliverables

Each student submits:

1. project-profile.md: identity, project title and weekly work schedule
2. topic-worksheet.md: candidate ideas, comparison and chosen scope
3. proposal-template.md: proposal v0, with all eight parts attempted

Templates are in labs/session2-solo/.

Each submission has one author and one project owner.

05. Lab – Your Topic

20 minutes: generate, compare and scope your own ideas

Set up your project profile

3 minutes: fill labs/session2-solo/project-profile.md.

Name and student ID

Working project title

Weekly progress-review time and coding blocks

Your current strengths and the skill you need to review

You will complete all development responsibilities yourself.

Individual brainstorming

5 minutes: worksheet part 1.

1. Write at least 6 problems from your own week.
2. Generate ideas before judging them.
3. Turn each idea into one user and one useful action.
4. Keep all ideas in your own worksheet.

Advice is welcome; topic selection and implementation remain individual.

Compare ideas with a scorecard

5 minutes: score three candidate topics from 0 to 2.

Criterion	0	1	2
User need	No clear user	Possible use	Clear recurring need
Python fit	Mostly unfamiliar	Some new work	Uses learned skills
MVP feasibility	Cannot finish alone	Scope uncertain	Runs by week 11

A high total cannot rescue a topic with feasibility 0.

For a tie, choose the smaller scope with accessible data.

Scope your chosen idea

7 minutes: worksheet part 3.

One-sentence goal: “For ___, my program helps ___ by ___.”

At most 3 Must features, each one line

At least 2 out-of-scope items

A named data source or small sample you can create

A fallback: what will you cut if the week-11 MVP is at risk?

Report your topic individually to the instructor.

06. Lab – Solo Proposal v0

25 minutes: draft, get feedback and revise

Draft the eight parts

18 minutes: labs/session2-solo/proposal-template.md.

Parts	Minutes
1–2. Problem and user	3
3–4. Features and scope	6
5–6. Data and skills	4
7. Milestones and work schedule	3
8. Risks	2

Attempt every part. Give unresolved decisions a date for follow-up.

Individual peer feedback

5 minutes: exchange drafts with one classmate.

1. Can the reader restate the goal in one sentence?

2. Is each Must feature testable?

3. Can one student finish the scope by week 11?

Give two strengths and one concern.

Review only: do not split implementation or create a joint submission.

Revise your own proposal after receiving feedback.

One-minute individual pitch

2 minutes within the lab: two volunteers, one minute each.

State the user’s problem.

Name your Must features.

Describe one observable demo result.

Listeners suggest one feature to cut and explain why.

If time is short, send your pitch as a written paragraph.

Common solo-project mistakes

A technology name instead of a user goal

Scope that needs several developers

Data access postponed until implementation

No time reserved for tests or documentation

A proposal filled with vague descriptions

Check questions

1. What makes a requirement testable? A clear pass/fail result.
2. What goes into the MVP? Must features only, at most 3 for this solo plan.
3. Why write Won't items? To record and defend scope decisions.
4. What does data design answer? What, who, format and amount.
5. Is joint development allowed? No. Each project is implemented individually.

Submit and prepare

Submit your profile, topic worksheet and solo proposal v0 via the LMS.

Read your own proposal before week 10 and explain your scope decisions.

Week 10: function decomposition, modules, data design and personal Git workflow.

Bring a laptop where git --version runs.

One student, one independently developed project and one individual demo.